

Equality in Dependent Type Theory: Lecture Notes

Elias Abou Farah and Arman Ahmed

November 17, 2025

Contents

1	Motivation and Overview	2
2	Equality as an Identity Type	2
2.1	Formation	2
2.2	Introduction (Reflexivity)	2
2.3	Elimination (J-rule) As Given in Lecture	2
2.4	Elimination (J -rule / equality induction)	3
2.5	Two derived tools: <code>ap</code> and <code>transport</code>	3
3	Worked Equalities	4
3.1	Successor preserves equality	4
3.2	Doubling equals adding a number to itself	4
3.3	A lecture example: if $x = \text{suc}(0)$ then $\text{double}(x) = \text{suc}(\text{suc}(0))$	4
3.4	Successor is injective (fully detailed)	5
3.5	Right identity of $+$ (expanded proof)	5
4	Intensional vs. Extensional Equality	5
5	Sigma Types and Existentials	5
5.1	Formation and judgmental equality	5
5.2	Introduction, Projections, and Elimination	6
5.3	Aside: a derived let-style eliminator (following NPS [5])	6
5.4	Existential specifications: the predecessor contract	7
5.5	Encoding negation via empty equalities	7
6	Historical and Research Perspectives	7
6.1	Historical notes	7
6.2	Current research threads	7
7	Common Patterns and Tactics	7
8	Practice Problems	8

1 Motivation and Overview

In dependent type theory we distinguish two notions of equality: *judgmental (definitional) equality*, written $M \equiv N$, and *propositional equality*, a first-class *type* written $M =_A N$.¹ Judgmental equality is part of the meta-theory (convertibility via computation rules) and requires no inhabitant. Propositional equality is internalized as a type whose terms are *witnesses* that two elements of the same type are equal.

This separation lets us *state and prove* equalities that are not definitionally true (e.g. $x + 0 = x$ for symbolic x) while retaining a decidable type-checking core.

Judgmental equality (informal recap)

Judgmental equality identifies terms up to β -reduction, η (if present), and the ι -rules of inductive eliminators. Two key meta-rules used implicitly throughout are:

- **Conversion:** if $\Gamma \vdash M : A$ and $A \equiv B$, then $\Gamma \vdash M : B$.
- **Congruence (example for arrows):** if $A \equiv A'$ and (under $x:A$) $B \equiv B'$, then $(A \rightarrow B) \equiv (A' \rightarrow B')$.

2 Equality as an Identity Type

We introduce an *identity type* (propositional equality) with the usual three rules: formation, introduction (reflexivity), and elimination (the *J*-rule), together with its computation rule.

2.1 Formation

$$\frac{\Gamma \vdash A : \text{Type} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : A}{\Gamma \vdash M =_A N : \text{Type}}.$$

2.2 Introduction (Reflexivity)

$$\frac{\Gamma \vdash M : A}{\Gamma \vdash \text{refl}_A(M) : M =_A M}.$$

We often write simply refl when A and M are clear.

2.3 Elimination (J-rule) As Given in Lecture

We first note a (admittedly very large) J-rule as we have formulated all of our structures:

$$\frac{\Gamma \vdash P : N_1 =_A N_2 \quad \Gamma \vdash N_1 : A \quad \Gamma \vdash N_2 : A \quad \Gamma, A : A \vdash M : [N_1/n_1, N_2/n_2, P/p]R \quad \Gamma, n_1 : A, n_2 : A, p : n_1 =_A n_2 \vdash R : u}{\Gamma \vdash \text{eqElim } P \text{ as } N_1 =_A N_2 \quad \text{return } (n_1, n_2, p.R) \quad \text{with } \text{refl}^A(n) \Rightarrow M}$$

At first look this is pretty incomprehensible, but let us try to break this down:

- P is the proof of equality that we would like to eliminate; specifically it shows that $n_1 = n_2$.

¹We follow standard presentations, e.g. [1, 2, 3].

- The type R is, in some sense, the proof we would like to produce, though *before* substituting the actual endpoints. We therefore require that

$$[N_1/n_1, N_2/n_2, P/p]R$$

is the type of the goal in the reflexive case.

- Since the only way to introduce an equality is with `refl`, we may pattern-match on P with the assumption $P \equiv \text{refl}^A(N_1)$.
- M is the branch of type R handling that reflexive case.
- The remaining premises ensure that each of these pieces is well typed.

2.4 Elimination (J -rule / equality induction)

Let P be a family $P(x, y, p)$ of types depending on $x:A$, $y:A$, and $p:x =_A y$. If we can prove the *reflexive case* for all x ,

$$d : \prod_{x:A} P(x, x, \text{refl}(x)),$$

then for any $a, b:A$ and any $p:a =_A b$ we obtain

$$J(d; a, b, p) : P(a, b, p).$$

This formulation is equivalent to the general `eqElim` rule from Section 2.3: there we eliminate a proof $P : N_1 =_A N_2$ by pattern matching on `refl` and supplying a branch M of type R . The clause $J(d; a, a, \text{refl}(a)) \equiv d(a)$ is exactly the reflexive case of `eqElim` specialized to the family $P(x, y, p)$, so both rules express the same dependent elimination principle for identity types.

Computation rule.

$$J(d; a, a, \text{refl}(a)) \equiv d(a),$$

i.e. eliminating a reflexivity proof computes to the corresponding branch.

2.5 Two derived tools: `ap` and `transport`

Lemma 2.1 (Action on paths). *Given $f:A \rightarrow B$ and $p:x =_A y$, there is $\text{ap}(f, p):f(x) =_B f(y)$.*

Proof. By J on p with motive $P(x, y, p) := f(x) =_B f(y)$; in the reflexive case we return $\text{refl}(f(x))$. □

Lemma 2.2 (Transport). *Let $P:A \rightarrow \text{Type}$. For $p:x =_A y$ there is a map*

$$\text{transport}^P(p) : P(x) \rightarrow P(y).$$

Proof. By J with motive $P'(x, y, p) := (P(x) \rightarrow P(y))$; in the reflexive case use the identity function. □

3 Worked Equalities

Lemma 3.1 (Right successor of $+$). *For all $n, m : \mathbb{N}$, $n + \text{suc}(m) =_{\mathbb{N}} \text{suc}(n + m)$.*

Proof. By induction on n .

Base. $n \equiv 0$. Then $0 + \text{suc}(m) \equiv \text{suc}(m) \equiv \text{suc}(0 + m)$ (judgmental equality) by the defining equations for $+$, so take *refl*.

Step. Assume $q : n + \text{suc}(m) =_{\mathbb{N}} \text{suc}(n + m)$. Using the recursive clause for $+$ we obtain $(\text{suc } n) + \text{suc}(m) \equiv \text{suc}(n + \text{suc}(m))$. Apply $\text{ap}(\text{suc}, q)$ to rewrite the inner $n + \text{suc}(m)$ to $\text{suc}(n + m)$, yielding $(\text{suc } n) + \text{suc}(m) =_{\mathbb{N}} \text{suc}(\text{suc}(n + m))$ as required. □

3.1 Successor preserves equality

Theorem 3.2. *For all $n, m : \mathbb{N}$, if $p : n =_{\mathbb{N}} m$ then $\text{ap}(\text{suc}, p) : \text{suc}(n) =_{\mathbb{N}} \text{suc}(m)$.*

Proof. Immediate from Lemma 2.1 with $f \equiv \text{suc}$. □

3.2 Doubling equals adding a number to itself

Theorem 3.3. *For all $n : \mathbb{N}$, $\text{double}(n) =_{\mathbb{N}} n + n$.*

Proof. By induction on n .

Base. $n \equiv 0$. Then $\text{double}(0) \equiv 0 \equiv 0 + 0$ by computation; take *refl*.

Step. Assume $q : \text{double}(n) =_{\mathbb{N}} n + n$. We must show $\text{double}(\text{suc } n) =_{\mathbb{N}} (\text{suc } n) + (\text{suc } n)$. Compute both sides:

$$\begin{aligned} \text{double}(\text{suc } n) &\equiv \text{suc}(\text{suc}(\text{double}(n))), \\ (\text{suc } n) + (\text{suc } n) &\equiv \text{suc}(n + \text{suc } n) = \text{suc}(\text{suc}(n + n)) \quad (\text{by Lemma 3.1}). \end{aligned}$$

Apply $\text{ap}(\lambda t. \text{suc}(\text{suc}(t)), q)$ to rewrite $\text{suc}(\text{suc}(\text{double}(n)))$ into $\text{suc}(\text{suc}(n + n))$, yielding the goal. □

3.3 A lecture example: if $x = \text{suc}(0)$ then $\text{double}(x) = \text{suc}(\text{suc}(0))$

Theorem 3.4. *For all $x : \mathbb{N}$, $x =_{\mathbb{N}} \text{suc}(0) \rightarrow \text{double}(x) =_{\mathbb{N}} \text{suc}(\text{suc}(0))$.*

Proof. Let $p : x =_{\mathbb{N}} \text{suc}(0)$. Use J with motive

$$P(n_1, n_2, p) \equiv \text{double}(n_1) =_{\mathbb{N}} \text{double}(n_2).$$

The reflexive case returns *refl* for all n . Thus we obtain

$$\text{double}(x) =_{\mathbb{N}} \text{double}(\text{suc}(0)).$$

By computation of *double*, $\text{double}(\text{suc}(0)) \equiv \text{suc}(\text{suc}(0))$. Conclude by conversion that $\text{double}(x) =_{\mathbb{N}} \text{suc}(\text{suc}(0))$. □

3.4 Successor is injective (fully detailed)

Theorem 3.5 (Successor injectivity). *For all $n, m : \mathbb{N}$, $\text{suc}(n) =_{\mathbb{N}} \text{suc}(m) \rightarrow n =_{\mathbb{N}} m$.*

Proof. Let $p : \text{suc}(n) =_{\mathbb{N}} \text{suc}(m)$. Apply Lemma 2.1 with $f \equiv \text{pred}$ to obtain

$$\text{ap}(\text{pred}, p) : \text{pred}(\text{suc}(n)) =_{\mathbb{N}} \text{pred}(\text{suc}(m)).$$

By definition of pred , both endpoints compute to n and m respectively, so conversion finishes the proof. $\square$

3.5 Right identity of $+$ (expanded proof)

Theorem 3.6. *For all $n : \mathbb{N}$, $n + 0 =_{\mathbb{N}} n$.*

Proof. Proceed by induction on n .

Base. $n \equiv 0$. Then $0 + 0 \equiv 0$ by definition, so take refl .

Step. Assume $q : n + 0 =_{\mathbb{N}} n$. Using the definition of $+$ we compute $(\text{suc } n) + 0 \equiv \text{suc}(n + 0)$. Apply Theorem 3.2 with $p \equiv q$ to rewrite $\text{suc}(n + 0)$ into $\text{suc}(n)$, finishing the step. $\square$

4 Intensional vs. Extensional Equality

In *intensional* Martin-Löf type theory, only the rules above are primitive; in particular there is no *equality reflection*

$$\frac{\Gamma \vdash p : M =_A N}{\Gamma \vdash M \equiv N : A},$$

and principles such as functional extensionality or univalence are not derivable without additional axioms [2, 3]. Adding equality reflection yields an *extensional* theory, but at the cost of complicating (and in general destroying) decidable type checking; standard proof assistants (Coq, Agda) adopt the intensional core.

5 Sigma Types and Existentials

The second lecture introduced the dependent pair (Σ) type, which internalizes *existential* statements. We keep the presentation close to [2].

5.1 Formation and judgmental equality

Given $A : \text{Type}$ and a family $B : A \rightarrow \text{Type}$, the dependent pair type

$$\Sigma_{x:A} B(x)$$

is itself a type. Its judgmental equality compares both components:

$$(\Sigma_{x:A} B(x)) \equiv (\Sigma_{x:A'} B'(x)) \quad \text{if } A \equiv A' \text{ and } B(x) \equiv B'(x) \text{ under } x:A.$$

The well-formedness side-conditions mirror those of Π -types: whenever $\Gamma \vdash A : \text{Type}$ and $\Gamma, x:A \vdash B(x) : \text{Type}$, we conclude $\Gamma \vdash \Sigma_{x:A} B(x) : \text{Type}$.

5.2 Introduction, Projections, and Elimination

An inhabitant of a dependent pair packages a witness and its dependent evidence:

$$\frac{\Gamma \vdash M : A \quad \Gamma \vdash N : B(M)}{\Gamma \vdash \langle M, N \rangle : \Sigma_{x:A} B(x)}.$$

Primitive projections. Following the presentation from lecture, the projections

$$\begin{aligned} \text{fst} &: \Sigma_{x:A} B(x) \rightarrow A, \\ \text{snd} &: \prod_{z:\Sigma_{x:A} B(x)} B(\text{fst}(z)), \end{aligned}$$

are taken as *primitive*. They satisfy the computation (β) rules:

$$\begin{aligned} \text{fst}(\langle M, N \rangle) &\equiv M, \\ \text{snd}(\langle M, N \rangle) &\equiv N. \end{aligned}$$

Elimination via projections. To use a term $z : \Sigma_{x:A} B(x)$, we eliminate it by reasoning with its components $\text{fst}(z)$ and $\text{snd}(z)$. Given a family $C(x, y)$ depending on $x:A$ and $y:B(x)$, producing a term

$$d(x, y) : C(x, y) \quad \text{in context } x:A, y:B(x),$$

yields a term

$$C(\text{fst}(z), \text{snd}(z))$$

when supplied with $z : \Sigma_{x:A} B(x)$. No separate eliminator is needed: elimination proceeds directly by projecting.

η -law. Every dependent pair is determined by its projections:

$$\langle \text{fst}(z), \text{snd}(z) \rangle \equiv z.$$

This expresses that a value of $\Sigma_{x:A} B(x)$ is judgmentally equal to the pair formed from its witness and evidence.

5.3 Aside: a derived let-style eliminator (following NPS [5])

Some presentations of dependent pairs, notably Nordström–Petersson–Smith [5], treat a let-style eliminator as primitive and define the projections from it. In the present notes we follow the lecture and take fst and snd as primitive, but a let-style eliminator can be *derived*.

Given a term

$$d(x, y) : C(x, y) \quad \text{in context } x:A, y:B(x),$$

and $z : \Sigma_{x:A} B(x)$, define the eliminator:

$$\text{split}(d, z) := d(\text{fst}(z), \text{snd}(z)).$$

It satisfies the expected β -law:

$$\text{split}(d, \langle M, N \rangle) \equiv d(M, N).$$

This derived operator corresponds to the primitive form used in Nordström–Petersson–Smith but is not taken as primitive here.

5.4 Existential specifications: the predecessor contract

With Σ available we can state existence properties cleanly. For example, the predecessor totality discussed in class is formulated as

$$\prod_{x:\mathbb{N}} (x =_{\mathbb{N}} \mathbf{0} \rightarrow \mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0})) \rightarrow \Sigma_{y:\mathbb{N}} \text{suc}(y) =_{\mathbb{N}} x,$$

i.e. every non-zero natural has a predecessor whose successor is propositionally equal to the original x . The proof follows the same inductive structure as our earlier recursor arguments: handle $x \equiv \mathbf{0}$ by contradiction, and in the successor case package the obvious witness $y \equiv n$ together with the reflexive equality proof. Erasing the proof-relevant pieces yields the expected computation for `pred` (this is the “program extraction” intuition highlighted in lecture).

5.5 Encoding negation via empty equalities

Although we have not introduced a primitive empty type, the class notes model falsity by the uninhabited equality $\mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0}) \equiv \mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0})$. Negation is then $A \rightarrow \mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0}) \equiv A \rightarrow \mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0})$. Because no term can inhabit $\mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0})$, any function of type $A \rightarrow \mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0})$ acts as a refutation of A . This trick suffices for small derivations (e.g. the “ $x \neq 0$ ” premise above) until we extend the core with an explicit empty type.

6 Historical and Research Perspectives

6.1 Historical notes

Identity types enter the literature with Martin-Löf’s 1970s formulations of constructive type theory, where equality witnesses replaced logical equivalence proofs [1]. The shift from external (judgmental) reasoning to internal identity proofs mirrors older distinctions in logic between definitional equality (dating back to Frege) and propositional equality (explicitly treated by Church). Modern expositions such as [2] highlight how J captures Leibniz’s indiscernibility principle inside the theory, making the rules simultaneously computational and proof-theoretic.

6.2 Current research threads

Homotopy Type Theory (HoTT) reinterprets $M =_A N$ as a space of paths, leading to univalence and higher inductive types [3]. A major line of current work studies computational presentations of these ideas; examples include cubical type theory, where paths are functions out of an abstract interval object and univalence becomes a definitional equality [4], and Cartesian cubical frameworks that scale to higher-dimensional proof assistants and synthetic homotopy constructions [6]. These developments aim to reconcile rich equality principles with canonicity and computation, and motivate the transport and `ap` patterns emphasized in these notes.

7 Common Patterns and Tactics

- Prefer definitional simplification first (unfold and compute). Use propositional rewriting only for the remaining differences.
- Choose J ’s motive so that the reflexive branch becomes `refl` (“the branch carries no extra information”).

- Use derived combinators: `ap` for function congruence; `transport` for dependent substitution.

8 Practice Problems

Practice 8.1 (Successor injectivity). Prove: $\forall n, m : \mathbb{N}. \text{suc}(n) =_{\mathbb{N}} \text{suc}(m) \rightarrow n =_{\mathbb{N}} m$. *Hint:* Use J on the given equality with a motive that peels off one `suc` on both sides.

Practice 8.2 (Left and right identity of $+$). (a) Prove by induction on x that $x + 0 = x$. (b) Prove that $0 + x = x$ (this one holds by computation with our definition).

Practice 8.3 (Successor on the right). Reprove Lemma 3.1 (the right-successor law for $+$) directly by induction on n .

Practice 8.4 (Symmetry of equality). Construct $\text{sym} : \prod_{x, y : \mathbb{N}} x =_{\mathbb{N}} y \rightarrow y =_{\mathbb{N}} x$ using the J -rule. *Hint:* Choose the motive $P(x, y, p) \equiv y =_{\mathbb{N}} x$ so that the reflexive branch reduces to `refl`.

Practice 8.5 (Transport preserves evenness). Define the predicate $\text{Even} : \mathbb{N} \rightarrow \text{Type}$ by $\text{Even}(0) \equiv \top$ and $\text{Even}(\text{suc}(\text{suc}n)) \equiv \text{Even}(n)$ (with no constructor for odd inputs). Using Lemma 2.2, prove that any $p : n =_{\mathbb{N}} m$ induces a map $\text{transport}^{\text{Even}}(p) : \text{Even}(n) \rightarrow \text{Even}(m)$.

Practice 8.6 (A variant of Theorem 3.4). Show: $\forall x : \mathbb{N}. x =_{\mathbb{N}} \text{suc}(0) \rightarrow \text{double}(x) =_{\mathbb{N}} 2$, where 2 is $\text{suc}(\text{suc}(0))$. Use the same motive as in Theorem 3.4.

Practice 8.7 (Sigma projections). Define `fst` and `snd` using the Σ -eliminator and prove their computation rules $\text{fst}(\langle M, N \rangle) \equiv M$ and $\text{snd}(\langle M, N \rangle) \equiv N$.

Practice 8.8 (Predecessor as an existential). Formalize the specification $\prod_{x : \mathbb{N}} (x =_{\mathbb{N}} \mathbf{0} \rightarrow \mathbf{0} =_{\mathbb{N}} \text{suc}(\mathbf{0})) \rightarrow \Sigma_{y : \mathbb{N}} \text{suc}(y) =_{\mathbb{N}} x$. Carry out the proof by recursion on x and explain where the “non-zero” hypothesis is used.

References

- [1] Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- [2] Robert Harper. *Practical Foundations for Programming Languages*. Cambridge University Press, 2nd edition, 2016.
- [3] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. <https://homotopytypetheory.org/book/>
- [4] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical Type Theory: A constructive interpretation of the univalence axiom. In *Proc. TYPES*, 2018.
- [5] Bengt Nordström, Kent Petersson, and Jan M. Smith. *Programming in Martin-Löf’s Type Theory: An Introduction*. Oxford University Press, 1990.
- [6] Carlo Angiuli, Kuen-Bang Hou, and Robert Harper. Cartesian cubical computational type theory: Constructive reasoning with paths and equalities. In D. R. Ghica, A. Jung (Eds.), *Computer Science Logic 2018, CSL 2018 Article 6* (Leibniz International Proceedings in Informatics, LIPIcs; Vol. 119). Schloss Dagstuhl- Leibniz-Zentrum für Informatik GmbH, Dagstuhl Publishing. <https://doi.org/10.4230/LIPIcs.CSL.2018.6>